5-DAY KUBERNETES BOOTCAMP · DAY 3 · MODULE 8

DaemonSets, StatefulSets & Jobs

A full hands-on lab: node-level agents, stateful applications with stable identity, and one-off / scheduled batch work

01
DaemonSets
Node agents, logging agents, monitoring agents

02
StatefulSets
Stateful apps, stable network identity, persistent storage

03
Jobs & CronJobs
Batch processing, scheduled tasks

Estimated time: 75–90 minutes

Level: Intermediate — builds on Pods, Deployments & Services from Day 2

⚠ **Lab cluster required**
You'll need a multi-node cluster (2+ nodes) to properly see DaemonSet scheduling behavior. A single-node kind/minikube cluster still works for StatefulSets and Jobs.

What This Lab Covers

So far you've worked with Deployments — workloads where "which Pod runs where" and "which Pod is which" don't matter, only the total count does. Today covers three workload types that exist specifically because that assumption breaks down for certain kinds of applications:

- **DaemonSets** — for workloads that must run on *every* node, exactly once each
- **StatefulSets** — for workloads where each Pod needs a stable, persistent identity
- **Jobs & CronJobs** — for work that runs to completion instead of running forever

> **WHY NOT JUST USE A DEPLOYMENT FOR EVERYTHING?**
> A Deployment's ReplicaSet doesn't care which node a Pod lands on, or whether a replacement Pod gets a new name and a new volume. That's perfect for stateless web servers — but wrong for a log collector that must run on every single node, or a database where losing track of "which replica is the primary" would be a real problem.

Workload type	Identity across restarts	Runs on	Typical use
Deployment	New name, new IP	Any N nodes	Stateless web apps, APIs
DaemonSet	New name, new IP	Every node (one each)	Log/metrics agents, CNI plugins
StatefulSet	Same name, same volume	Any N nodes	Databases, queues, clustered apps
Job / CronJob	Runs once, then exits	Any node	Batch jobs, scheduled tasks

Prerequisites

- A running Kubernetes cluster with `kubectl` configured (ideally 2+ nodes)
- A default `StorageClass` available for dynamic volume provisioning (check with `kubectl get storageclass`)
- Comfort with basic `kubectl apply` / `get` / `describe` / `logs` workflow

Module 1 — DaemonSets

Node agents · Logging agents · Monitoring agents

The Concept

A **DaemonSet** guarantees that a copy of a Pod runs on every node in the cluster — no more, no less. Add a new node, and the DaemonSet automatically schedules a Pod onto it. Remove a node, and that Pod goes with it. You never set a replica count; the node count *is* the replica count.

> **THINK OF IT LIKE A BUILDING'S FLOOR SECURITY GUARD**
>
> Instead of hiring a fixed number of guards and letting a building manager scatter them anywhere (a Deployment), a DaemonSet is like a policy: "*post exactly one guard on every floor.*" Add a new floor to the building (a new node joins the cluster) and a guard is automatically posted there. Demolish a floor (a node is removed) and that guard's post goes away with it.

Real-World Examples

Category	Real tool	Why it must be a DaemonSet
Logging agent	Fluent Bit / Fluentd	Needs to read `/var/log` on every node to ship every node's logs
Monitoring agent	Prometheus Node Exporter	Exposes hardware/OS metrics that only exist locally on each node
Networking	Calico / Weave / kube-proxy	Every node needs its own copy to route traffic correctly
Security	Datadog Agent / Falco	Runtime security scanning must observe every node's processes

LAB 1.1 — Deploy a Logging Agent DaemonSet

Create `logging-daemonset.yaml`:

```yaml
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluentbit-agent
  labels:
    app: fluentbit-agent
spec:
  selector:
    matchLabels:
      app: fluentbit-agent
  template:
    metadata:
      labels:
        app: fluentbit-agent
    spec:
      tolerations:
      - key: node-role.kubernetes.io/control-plane
        effect: NoSchedule
      containers:
      - name: fluent-bit
        image: fluent/fluent-bit:2.2
        resources:
          requests:
            cpu: "50m"
            memory: "64Mi"
        volumeMounts:
        - name: varlog
          mountPath: /var/log
      volumes:
      - name: varlog
        hostPath:
          path: /var/log
```

Deploy and verify one Pod landed on every node:

```
$ kubectl apply -f logging-daemonset.yaml

$ kubectl get pods -l app=fluentbit-agent -o wide
# Expected: one Pod per node, each on a different NODE column
NAME                    READY   STATUS    NODE
fluentbit-agent-4kx2p   1/1     Running   worker-1
fluentbit-agent-9j7qm   1/1     Running   worker-2
fluentbit-agent-p2vln   1/1     Running   control-plane

$ kubectl get nodes --no-headers | wc -l   # compare to pod count
```

NOTE THE `TOLERATIONS` BLOCK

By default, control-plane nodes have a *taint* that repels normal Pods. A logging agent needs to run *everywhere*, including the control plane — the toleration above lets it schedule there too.

LAB 1.2 — Deploy a Monitoring Agent DaemonSet

Create `monitoring-daemonset.yaml`:

```yaml
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-exporter
spec:
  selector:
    matchLabels:
      app: node-exporter
  template:
    metadata:
      labels:
        app: node-exporter
    spec:
      hostNetwork: true
      hostPID: true
      containers:
      - name: node-exporter
        image: prom/node-exporter:v1.7.0
        ports:
        - containerPort: 9100
        resources:
          requests:
            cpu: "50m"
            memory: "30Mi"
```

```
$ kubectl apply -f monitoring-daemonset.yaml
$ kubectl get pods -l app=node-exporter -o wide

# Query metrics directly from one node's exporter
$ kubectl exec -it <node-exporter-pod> -- wget -qO- localhost:9100/metrics | head -5
```

`HOSTNETWORK` AND `HOSTPID`

Node-level agents often need `hostNetwork: true` (to bind directly to the node's network) and `hostPID: true` (to see host processes). These grant real access to the underlying node — only use them for trusted, purpose-built agents, never for regular application workloads.

VERIFY — Scheduling Behavior

```
# Cordon a node so it can't accept new Pods, then delete its agent Pod
$ kubectl cordon worker-2
$ kubectl delete pod -l app=fluentbit-agent --field-selector spec.nodeName=worker-2

# The DaemonSet controller tries to reschedule immediately —
# but a cordoned node refuses new Pods, so it stays Pending until uncordoned
$ kubectl get pods -l app=fluentbit-agent -o wide

$ kubectl uncordon worker-2
# Pod schedules back onto worker-2 within seconds
```

Module 2 — StatefulSets

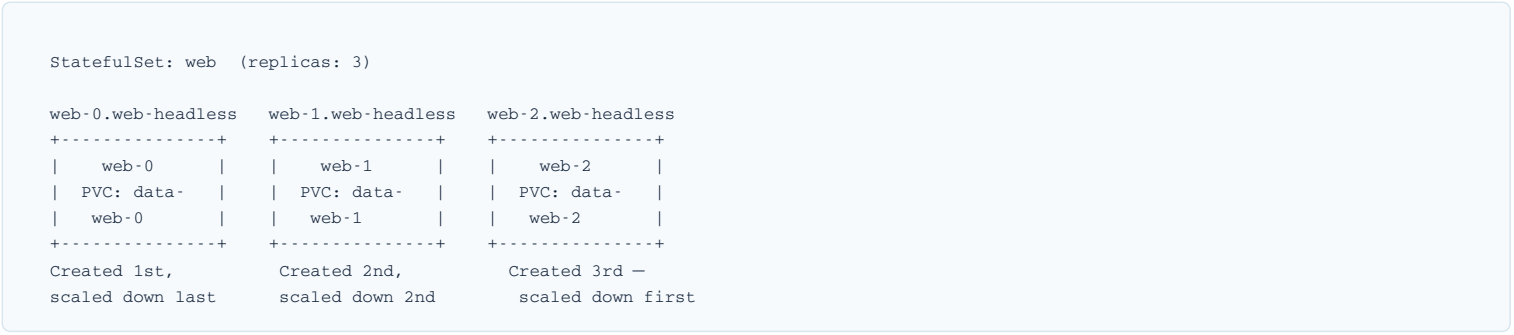
Stateful applications · Stable network identity · Persistent storage

The Concept

A **StatefulSet** manages Pods that need a durable identity. Each Pod gets a predictable name (`web-0` , `web-1` , `web-2` ...), a stable DNS hostname that follows it even after a restart, and — if you use `volumeClaimTemplates` — its own dedicated, persistent volume that survives Pod deletion and gets re-attached to whichever Pod takes that ordinal's place.

THINK OF IT LIKE RESERVED, NUMBERED PARKING SPOTS

A Deployment is like open parking — any car (Pod) can use any open spot, and it doesn't matter which. A StatefulSet is like assigned parking: spot **#0** is always spot #0, with the same nameplate and the same storage locker attached, no matter which specific car is parked there today. Swap the car (recreate the Pod) and the new one still inherits spot #0's identity and locker contents.



Real-World Examples

Application	Why it needs a StatefulSet
Kafka / RabbitMQ clusters	Each broker has an identity peers rely on (broker-0, broker-1...)
Cassandra / Elasticsearch	Each node owns a specific data shard on its own persistent disk
PostgreSQL / MySQL replica sets	The "primary" vs "replica" role is tied to a specific, stable Pod
ZooKeeper	Consensus protocols require every peer to have a fixed, known address

LAB 2.1 — Headless Service (Required for Stable DNS)

StatefulSets need a **headless Service** (`clusterIP: None`) to hand out per-Pod DNS names instead of one shared load-balanced IP. Create `web-headless-svc.yaml` :

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web-headless
spec:
  clusterIP: None
  selector:
    app: web
  ports:
  - port: 80
    name: http
```

LAB 2.2 — The StatefulSet

Create `web-statefulset.yaml` :

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: web
spec:
  serviceName: web-headless
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
      - name: nginx
        image: nginx:1.27
        ports:
        - containerPort: 80
        volumeMounts:
        - name: data
          mountPath: /usr/share/nginx/html
  volumeClaimTemplates:
  - metadata:
      name: data
    spec:
      accessModes: ["ReadWriteOnce"]
      resources:
        requests:
          storage: 1Gi
```

```
$ kubectl apply -f web-headless-svc.yaml
$ kubectl apply -f web-statefulset.yaml

$ kubectl get pods -l app=web
# Notice: created in strict order, one at a time
NAME    READY   STATUS
web-0   1/1     Running
web-1   1/1     Running
web-2   1/1     Running

$ kubectl get pvc
# Expected: one PVC per Pod, matching its ordinal
NAME          STATUS   VOLUME     CAPACITY
data-web-0    Bound    pvc-a1b2   1Gi
data-web-1    Bound    pvc-c3d4   1Gi
data-web-2    Bound    pvc-e5f6   1Gi
```

VERIFY — Stable Network Identity

```
# Resolve each Pod's own stable DNS name from inside the cluster
$ kubectl run dns-test --rm -it --image=busybox --restart=Never -- \
    nslookup web-0.web-headless.default.svc.cluster.local

# Expected: resolves directly to web-0's Pod IP — not a load-balanced VIP
```

VERIFY — Persistent Storage Survives Pod Deletion

```
# Write a file into web-0's persistent volume
$ kubectl exec web-0 -- sh -c 'echo "hello from web-0" > /usr/share/nginx/html/proof.txt'

# Delete the Pod entirely
$ kubectl delete pod web-0

# StatefulSet recreates it — SAME name, SAME PVC re-attached
$ kubectl get pods -l app=web
# web-0 is back, freshly created but re-using pvc data-web-0

$ kubectl exec web-0 -- cat /usr/share/nginx/html/proof.txt
# Expected output: hello from web-0   <-- the file survived!
```

COMPARE TO A DEPLOYMENT

Try the same delete-and-check on a regular Deployment Pod — the replacement Pod gets a brand-new name and, if you weren't using a StatefulSet, an entirely empty volume. That difference is the entire point of a StatefulSet.

Module 3 — Jobs & CronJobs

Batch processing · Scheduled tasks

The Concept

A **Job** runs a Pod until it successfully completes, then stops — it doesn't restart the container in a loop forever the way a Deployment does. A **CronJob** is a Job that gets created automatically on a recurring schedule, using standard cron syntax.

> **THINK OF IT LIKE HIRING MOVERS VS. SCHEDULING TRASH PICKUP**
>
> A **Job** is like hiring a moving company for a single move: the truck shows up, does the work exactly once, and leaves — it's not a tenant living in the building forever (that's a Deployment). A **CronJob** is like the building's recycling truck: it shows up automatically every Tuesday at 7am on a fixed schedule, whether or not anyone remembers to call it.

Real-World Examples

Type	Example	Why "run once" or "on schedule" fits
Job	Database schema migration	Must run exactly once per deploy, then stop
Job	Batch image/video processing	Process a queue of files once, exit when done
CronJob	Nightly database backup	Needs to run automatically at 2am, every day
CronJob	Report generation / cache cleanup	Recurring maintenance on a fixed schedule

LAB 3.1 — A Batch Processing Job

Create `pi-job.yaml` — a Job that computes π to 2000 digits, representing a one-off compute task:

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: pi-calc
spec:
  completions: 1
  backoffLimit: 3
  template:
    spec:
      containers:
      - name: pi
        image: perl:5.34
        command: ["perl", "-Mbignum=bpi", "-wle", "print bpi(2000)"]
      restartPolicy: Never
```

```
$ kubectl apply -f pi-job.yaml

$ kubectl get jobs
NAME       COMPLETIONS   DURATION
pi-calc    1/1           8s

$ kubectl logs job/pi-calc
# Expected: 2000 digits of pi printed, then the Pod exits successfully
```

> **BACKOFFLIMIT**
>
> If the container fails, the Job retries up to `backoffLimit` times before giving up and marking itself Failed — this is Kubernetes' built-in retry logic for batch work, something a plain Pod doesn't have.

Optional: Parallel Jobs

For a queue of independent work items, run several Pods at once by adding `parallelism`:

```yaml
spec:
  completions: 5
  parallelism: 2   # up to 2 Pods running at a time, until 5 total succeed
```

LAB 3.2 — A Scheduled CronJob

Create `backup-cronjob.yaml` — simulating a nightly backup task, scheduled every 2 minutes for the lab:

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: nightly-backup
spec:
  schedule: "*/2 * * * *"   # every 2 min (prod: "0 2 * * *" = 2am daily)
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 1
  jobTemplate:
    spec:
      template:
        spec:
          containers:
          - name: backup
            image: busybox
            command:
            - sh
            - -c
            - "echo Backing up data at $(date); sleep 5; echo Backup complete"
          restartPolicy: OnFailure
```

```
$ kubectl apply -f backup-cronjob.yaml
$ kubectl get cronjobs

# Watch it trigger automatically — wait ~2 minutes
$ kubectl get jobs --watch

# Inspect the most recent run's logs
$ kubectl logs job/$(kubectl get jobs -o jsonpath='{.items[-1].metadata.name}')
# Expected: "Backing up data at ...", then "Backup complete"
```

Field	Purpose
`schedule`	Standard 5-field cron syntax: minute, hour, day-of-month, month, day-of-week
`concurrencyPolicy: Forbid`	Skip a new run if the previous one is still going — prevents overlapping backups
`successfulJobsHistoryLimit`	How many completed Job records to keep around for inspection
`startingDeadlineSeconds`	(optional) How late a missed run is allowed to start before being skipped entirely

REAL-WORLD LESSON

A payments company once had a nightly reconciliation CronJob whose runs occasionally overlapped under load, double-processing records. Setting `concurrencyPolicy: Forbid` is a one-line fix for exactly that class of bug — always set it explicitly rather than relying on the default (`Allow`).

Cleanup

```
$ kubectl delete daemonset fluentbit-agent node-exporter
$ kubectl delete statefulset web
$ kubectl delete service web-headless
$ kubectl delete pvc -l app=web
$ kubectl delete job pi-calc
$ kubectl delete cronjob nightly-backup
```

STATEFULSET PVCS AREN'T DELETED AUTOMATICALLY

Deleting a StatefulSet does **not** delete its PersistentVolumeClaims by default — this is intentional, so you don't lose a database's data by accident. Clean them up explicitly, as shown above, once you're sure you no longer need them.

Key Takeaways

Workload	One-line summary
DaemonSet	Exactly one Pod per node, automatically — for agents that must observe every node
StatefulSet	Stable name + stable DNS + stable storage per Pod — for apps that remember who they are
Job	Runs to completion once, with built-in retries — for one-off batch work
CronJob	A Job created automatically on a cron schedule — for recurring, unattended tasks